

Universidade de Brasília
Faculdade de Tecnologia
Departamento de Engenharia Civil e Ambiental

Implementação do Método da Rigidez Direta

Trabalho II

Disciplina: Teoria das Estruturas II

Tarso de Matos Bessa (22/2013950)

Brasília, DF
Novembro de 2026

Sumário

1. Introdução 1
   1.1. Linguagem 1
2. Arquitetura do Software 1
   2.1. Organização do diretório 2
   2.2. Fluxo da entrada de dados 2
   2.3. Fluxo de Execução 3
3. Decisões de Projeto e Implementação 5
   3.1. Definição e Leitura da Entrada de Dados 5
   3.2. Estratégia de Montagem da Rigidez Global 5
   3.3. Tipos Concretos e Desempenho 6
   3.4. Estrutura do Elemento 6
   3.5. Tratamento das Condições de Contorno e Carregamentos 7
      3.5.1. Condições de contorno 7
      3.5.2. Cargas nodais 7
      3.5.3. Cargas distribuídas 7
   3.6. Resolução do Sistema 8
   3.7. Cálculo dos Esforços Internos 9
4. Validação dos Modelos Estruturais 9
   4.1. Treliça Bidimensional 10
   4.2. Treliça Espacial 11
   4.3. Viga de Euler-Bernoulli 12
   4.4. Pórtico Bidimensional 13
5. Melhorias Futuras 15
6. Conclusão 15

1. Introdução

O Método da Rigidez Direta é a base dos programas de análise estrutural. A estrutura é dividida em elementos unidimensionais, cada um com a sua matriz de rigidez. Essas matrizes são somadas em uma matriz global K, e o comportamento da estrutura passa a ser descrito por $KU = F$. Resolvido o sistema, obtêm-se as reações de apoio e os esforços internos de cada barra.

No Trabalho I, o método foi aplicado manualmente. Este trabalho implementa um programa que lê a descrição de uma estrutura em um arquivo de entrada e devolve deslocamentos, reações e esforços internos.

O escopo são treliças (planas e espaciais), vigas de Euler-Bernoulli e pórticos planos, com material elástico linear e pequenos deslocamentos. Cargas distribuídas uniformes são aceitas em vigas (barras horizontais) e em barras de pórtico de qualquer orientação, informadas em eixos globais.

1.1. Linguagem

O programa foi escrito em Julia, pelos seguintes motivos:

- sintaxe simples e código compilado com desempenho próximo ao de C;
- suporte nativo a matrizes, a matrizes esparsas e à resolução de sistemas lineares (operador \);
- índices começando em 1, de modo que o número do nó ou do grau de liberdade coincide com a sua posição nas matrizes, como nas deduções manuais.

Usam-se apenas bibliotecas padrão: `LinearAlgebra`, `SparseArrays`, `TOML` (leitura da entrada) e `Printf` (relatório).

2. Arquitetura do Software

O programa é um único módulo Julia, `DirectStiffness`, dividido em arquivos pequenos, cada um responsável por uma etapa da análise. Os dados de entrada ficam em arquivos TOML, e os resultados são gravados em arquivos de texto. O código-fonte está disponível em https://github.com/bessa-t/Direct-Stifness.

2.1. Organização do diretório

```
Direct-Stifness/
├── inputs/                      arquivos de entrada (TOML)
│   ├── 2d_truss.toml
│   ├── 2d_beam.toml
│   ├── 2d_frame.toml
│   └── ...
├── outputs/                     relatórios gerados (.txt)
├── relatorio/                   este documento (Typst)
└── src/
    ├── Direct_Stiffness.jl      módulo; único arquivo com include
    ├── input.jl                 leitura do TOML e montagem da estrutura
    ├── assemble.jl              Structure e matriz de rigidez global
    ├── boundary.jl              vetor de deslocamentos prescritos
    ├── external_forces.jl       vetor de forças e cargas equivalentes
    ├── solver.jl                solução do sistema por partição
    ├── internal_forces.jl       esforços internos por barra
    ├── report.jl                relatório em texto
    ├── test.jl                  roda o exemplo e imprime na tela
    ├── types/
    │   ├── material_section.jl  material e seções
    │   ├── geometry.jl          comprimento e cossenos diretores
    │   └── element.jl           tipo Element e GDL do elemento
    └── elements/
        ├── truss.jl             treliça
        ├── bernoulli_beam.jl    viga de Euler-Bernoulli
        └── frame.jl             pórtico plano
```

Figura 1: Organização dos arquivos do projeto.

Os arquivos seguem três critérios:

- **Tipos de dados** (`types/`): definem os tipos usados no programa (material, seção, geometria, elemento) e não contêm lógica de análise.
- **Elementos** (`elements/`): um arquivo por tipo de elemento estrutural, com a sua matriz de rigidez e a sua matriz de transformação, com exceção ao elemento de viga, que é dado como sempre horizontal.
- **Etapas da análise** (raiz de `src/`): um arquivo por etapa do processamento, na ordem em que acontecem (leitura, com apoios e cargas; montagem; solução; esforços; relatório).

Só o arquivo `Direct_Stiffness.jl` faz `include` dos demais.

2.2. Fluxo da entrada de dados

A leitura do arquivo TOML está concentrada em `input.jl`: é o único arquivo que conhece os nomes das chaves e o formato das linhas. Todo o restante do programa recebe dados já validados, na forma de estruturas e vetores. A função `load_model` devolve a estrutura `s`, o vetor de deslocamentos prescritos `u_pre`, o vetor de forças `F`, as suas parcelas `f_vec` (cargas nodais) e `f_dist` (cargas distribuídas) e a carga distribuída de cada barra, `element_q`.

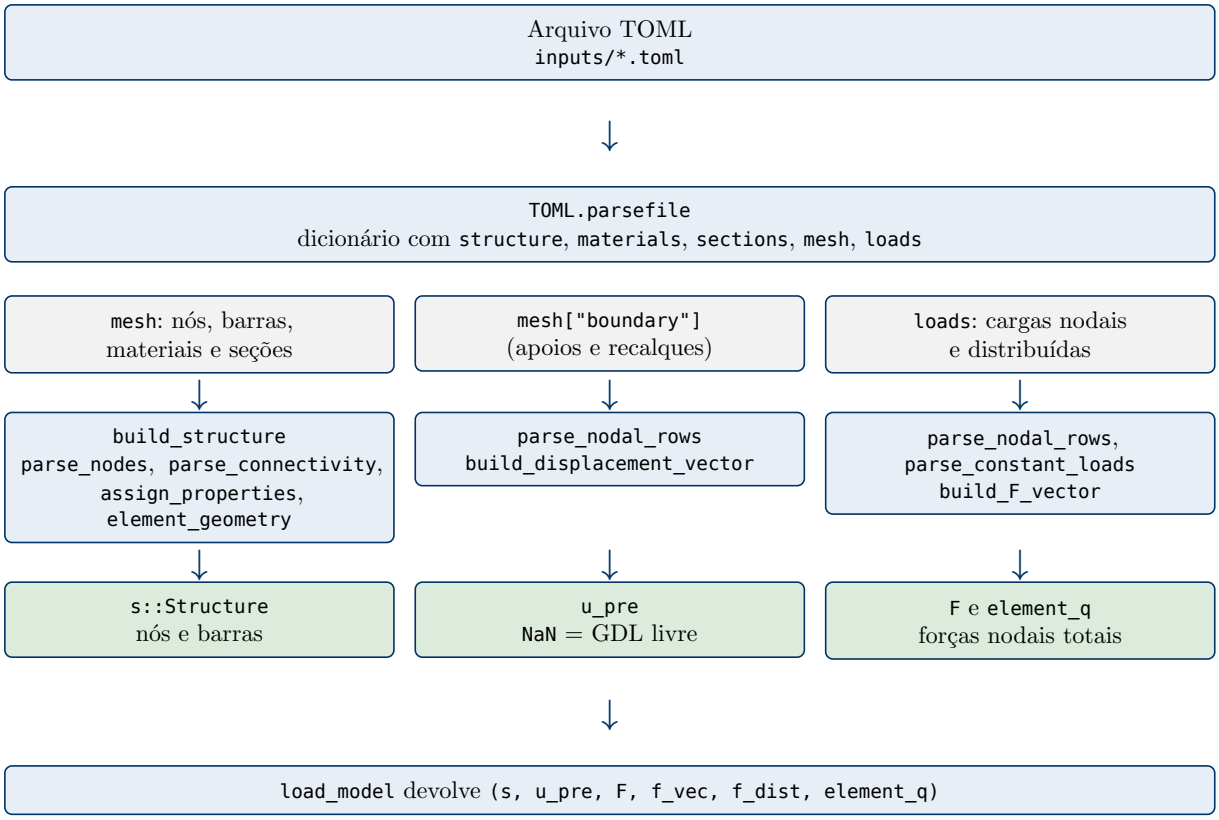


Figura 2: Fluxograma da entrada de dados.

Cada ramo do fluxograma valida o que lê: existência dos nós citados, número de valores por nó compatível com os graus de liberdade, barras com dois nós existentes e existência de pelo menos um apoio. Erros de entrada interrompem a execução com uma mensagem indicando o que está errado.

2.3. Fluxo de Execução

A análise completa é executada pela função `run_analysis`, definida em `Direct_Stiffness.jl`. Ela encadeia as cinco etapas do método, cada uma implementada em um arquivo próprio, e grava o relatório em `outputs/<nome do arquivo de entrada>.txt`. Os dados passam de uma etapa para a seguinte apenas por argumentos e valores de retorno, sem variáveis globais.

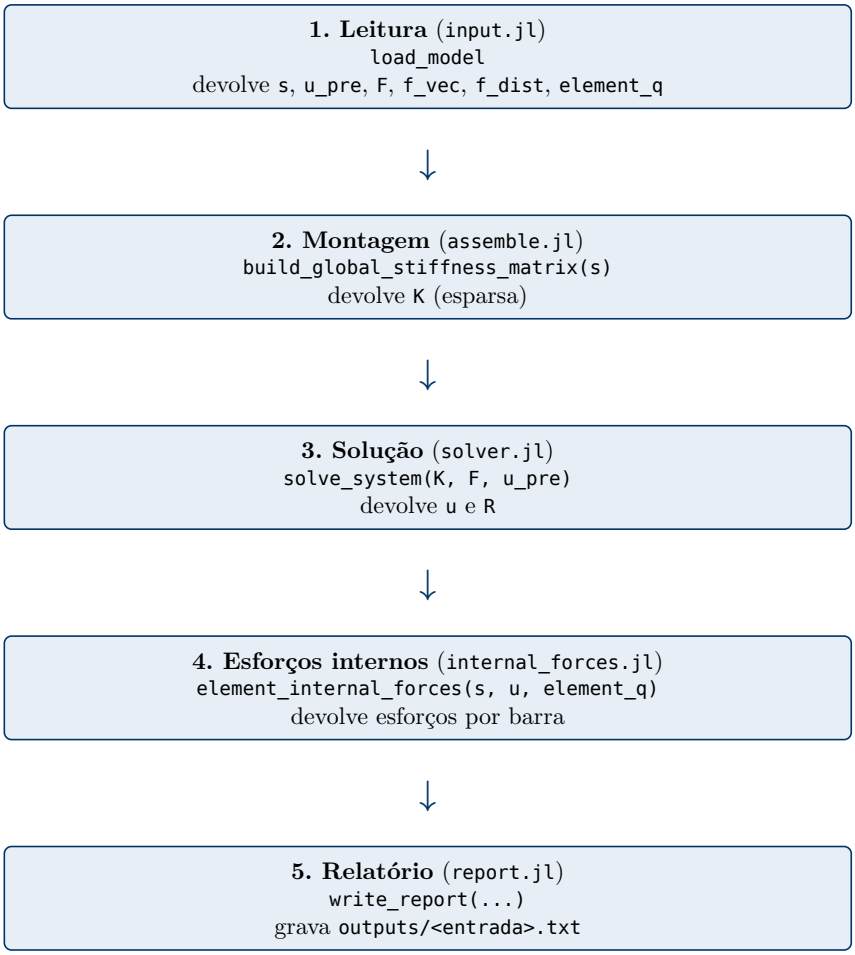


Figura 3: Fluxo de execução do programa.

1. Leitura. O arquivo TOML é convertido em uma estrutura `Structure`, no vetor de deslocamentos prescritos `u_pre` e no vetor de forças `F` (Figura 2). Os apoios e as cargas só servem para montar esses vetores, e depois deixam de existir como objetos.

2. Montagem. Cada barra calcula a sua matriz de rigidez já em eixos globais, e esta é somada na posição dos seus graus de liberdade na matriz global esparsa `K`.

3. Solução. Os graus de liberdade prescritos são eliminados do sistema, e resolve-se $K_{ll}U_l = F_l - K_{lp}U_p$ apenas para os graus de liberdade livres, em que l indica livre e p prescrito. Em seguida, as reações são obtidas por $R = KU - F$, tomadas nos graus de liberdade prescritos.

4. Esforços internos. Para cada barra, os deslocamentos dos seus nós são lidos do vetor global `u`. Eles são levados para os eixos locais da barra, e deles saem o esforço normal, o cortante e o momento fletor.

5. Relatório. Modelo, cargas, deslocamentos prescritos, deslocamentos nodais, reações e esforços internos são escritos em texto plano.

O programa pode ser executado de duas formas. A primeira grava o relatório do pórtico em `outputs/2d_frame.txt`, e a segunda imprime na tela o relatório do exemplo fixo `inputs/2d_truss.toml`:

```
julia src/Direct_Stiffness.jl inputs/2d_frame.toml
julia src/test.jl
```

3. Decisões de Projeto e Implementação

3.1. Definição e Leitura da Entrada de Dados

Escolheu-se o formato TOML porque é fácil de ler e de escrever, tem leitor nativo em Julia e dispensa um analisador próprio: `TOML.parsefile` devolve diretamente um dicionário. O exemplo abaixo descreve o pórtico do Exercício IV.

```
structure = "2d_frame"

[materials]
steel = { E = 180e6, v = 0.3 }          # E [kN/m2]

[sections]
sec_1 = { A = 5.0e-3, Iz = 2.1e-6 }     # A [m2], Iz [m4]

[mesh]
default_material = "steel"
default_section = "sec_1"
nodes = [[0.0, 0.0], [0.0, 2.0], [2.0, 2.0], [2.0, 0.0]]
elements = [[1, 2], [2, 3], [3, 4]]     # nó i, nó j
boundary = [
    [1, 0.0, 0.0, 0.0],                 # nó, ux, uy, θ (nan = livre)
    [4, 0.0, 0.0, 0.0],
]

[loads]
constant_loads = [[2, 0.0, -0.5]]       # elemento, qx, qy [kN/m]
```

Nós e barras são numerados pela ordem das linhas. O tipo de estrutura (`structure`) escolhe, em tabelas de consulta, quantos graus de liberdade (GDL) cada nó tem e como a seção é lida. Acrescentar um tipo novo é acrescentar uma entrada em cada tabela. O arquivo `input.jl` converte o dicionário nas estruturas e vetores usados pelo restante do programa.

3.2. Estratégia de Montagem da Rigidez Global

Os GDL são numerados por nó: o nó n ocupa as posições $(n-1)d+1$ a nd, em que d é o número de GDL por nó (2 na treliça plana e na viga, 3 na treliça espacial e no pórtico plano). Cada elemento calcula a sua matriz de rigidez já em eixos globais, e a matriz global é montada de forma esparsa, por triplas (linha, coluna, valor):

```
function build_global_stiffness_matrix(s::Structure)
    d = s.dofs_per_node             # GDL por nó
    n = nnodes(s) * d               # tamanho da matriz

    rows = Int[]; cols = Int[]; vals = Float64[]

    for e in s.elements
        idx = element_dofs(e, d)    # GDL globais do elemento
        Ke  = element_stiffness(e)  # rigidez em eixos globais

        for b in eachindex(idx), a in eachindex(idx)
            push!(rows, idx[a])     # linha global
            push!(cols, idx[b])     # coluna global
            push!(vals, Ke[a, b])   # valor
        end
    end

    return sparse(rows, cols, vals, n, n)   # posições repetidas são somadas
end
```

A função `sparse` soma os valores de posições repetidas, que é exatamente a soma da montagem. Assim evita-se alocar uma matriz densa $n \times n$, cuja memória cresce com o quadrado do número de GDL.

3.3. Tipos Concretos e Desempenho

Todos os `struct` do programa têm campos com tipo concreto: `Float64` para propriedades e coordenadas, `Int` para números de nós, `Tuple{Int,Int}` para os nós de uma barra e `Vector{Vector{Float64}}` para as coordenadas da estrutura. O elemento (`Element{G,S}`) e a estrutura (`Structure{E}`) são paramétricos, de modo que o compilador conhece os tipos exatos de cada elemento, e o despacho de `element_stiffness` e `transformation_matrix` é resolvido na compilação. Assim o Julia gera código de máquina específico para cada tipo, sem verificações de tipo em tempo de execução.

Essa escolha foi feita pensando em desempenho, mas nenhuma medição de tempo foi realizada, pois os exemplos são pequenos. Duas exceções conhecidas poderiam ser revistas em malhas grandes: `element_geometry` devolve `GeomData2D` ou `GeomData3D` conforme a dimensão dos nós (tipo de retorno não fixo), e a montagem acumula as triplas com `push!`, sem alocar os vetores com antecedência.

3.4. Estrutura do Elemento

`Element{G,S}` guarda a geometria (`G`), o material e a seção (`S`) de uma barra. A geometria é `GeomData2D` ou `GeomData3D`, conforme a dimensão dos nós, e já traz pronto o que a rigidez e a transformação precisam (nós, cossenos diretores, comprimento):

```
struct Element{G<:AbstractGeom, S<:AbstractSection}
    geometry :: G
    material :: Material
    section  :: S
end

struct GeomData2D <: AbstractGeom
    nodes::Tuple{Int,Int}
    c::Float64   # cosseno diretor
    s::Float64   # seno diretor
    L::Float64   # comprimento
end
```

Cada tipo de elemento é só um alias de `Element` fixando `G` e `S`, o que dá despacho estático: o pórtico plano (`Frame_2D`) usa `GeomData2D` e `Frame_2D_Section` (A e I_z):

```
const Frame_2D = Element{GeomData2D, Frame_2D_Section}

function element_stiffness(e::Frame_2D)
    E, A, L, I = e.material.E, e.section.A, e.geometry.L, e.section.Iz
    ...
end
```

3.5. Tratamento das Condições de Contorno e Carregamentos

Apoios e cargas são lidos do TOML e viram dois vetores com um valor por GDL: `u_pre` (deslocamentos prescritos) e `F` (forças). A tabela resume o formato de cada dado no pórtico plano. Nos demais elementos, apoios e cargas nodais têm um valor para cada GDL do nó:

Dado	**Linha no TOML**	**Efeito**
Apoio ou recalque	`[nó, ux, uy, θ]`	Preenche `u_pre`: valor prescrito (0 = apoio) ou `nan` (GDL livre)
Carga nodal	`[nó, Fx, Fy, M]`	Somada em `F`, nos GDL do nó
Carga distribuída	`[elemento, qx, qy]`	Forças nodais equivalentes, somadas em `F`

Tabela 1: Formato dos apoios e das cargas.

3.5.1. Condições de contorno

Um apoio que trava só em y no nó 1 é escrito `[1, nan, 0.0]`, e um recalque é um valor diferente de zero. Os índices de GDL livres e prescritos saem de `findall(isnan, u_pre)` e `findall(!isnan, u_pre)`.

3.5.2. Cargas nodais

As forças e momentos aplicados diretamente nos nós são somados em `F` nos GDL do nó (`F[node_dofs(nó, d)] .+= valores`).

3.5.3. Cargas distribuídas

As linhas `[elemento, qx, qy]` dão uma carga uniforme por comprimento de barra, em eixos globais. Ela é convertida em forças nodais equivalentes pela função `equivalent_nodal_loads`, que tem um método para cada elemento.

Viga. Como é horizontal e não tem GDL axial, basta o vetor de engastamento perfeito:

```
function equivalent_nodal_loads(e::Beam, qx, qy)
    L = e.geometry.L
    return [qy*L/2, qy*L^2/12, qy*L/2, -qy*L^2/12]
end
```

Pórtico. A carga é decomposta nas componentes ao longo da barra (q_a) e perpendicular a ela (q_t). As forças equivalentes são calculadas nos eixos locais e levadas aos eixos globais com a transposta da matriz de transformação, a mesma usada na montagem:

```
function local_load(e, qx, qy)
    c, s = e.geometry.c, e.geometry.s
    return (qa = c*qx + s*qy, qt = -s*qx + c*qy)
end

function equivalent_nodal_loads(e::Frame_2D, qx, qy)
    L = e.geometry.L
    qa, qt = local_load(e, qx, qy)
    f_local = [qa*L/2, qt*L/2, qt*L^2/12, qa*L/2, qt*L/2, -qt*L^2/12]
    return transformation_matrix(e)' * f_local    # eixos globais
end
```

O resultado é somado em `F`. Como a reação é calculada por $R = KU - F$, as cargas distribuídas não precisam de tratamento separado.

3.6. Resolução do Sistema

Reordenando os GDL em livres (l) e prescritos (p), o sistema fica:

$$\begin{pmatrix} K_{ll} & K_{lp} \\ K_{pl} & K_{pp} \end{pmatrix} \begin{pmatrix} U_l \\ U_p \end{pmatrix} = \begin{pmatrix} F_l \\ F_p + R_p \end{pmatrix}$$

Como U_p é conhecido, os deslocamentos livres saem de

$$K_{ll} U_l = F_l - K_{lp} U_p$$

No código, `free` e `prescribed` guardam os índices obtidos por `findall`, `rhs` é o lado direito, e o operador `\` do Julia escolhe o método de solução.

```
function solve_system(K, F, u_pre)
    free       = findall(isnan, u_pre)
    prescribed = findall(!isnan, u_pre)

    Kff = K[free, free]
    rhs = F[free] - K[free, prescribed] * u_pre[prescribed]

    uf = Kff \ rhs

    u = copy(u_pre)
    u[free] = uf

    R = K * u - F          # reações: R[prescribed]
    return u, R
end
```

Com todos os deslocamentos conhecidos, $R = KU - F$ vale zero nos GDL livres (equilíbrio) e dá a reação de apoio nos GDL prescritos. Nesses GDL, F contém apenas as cargas conhecidas (F_p), como as forças equivalentes de uma carga distribuída em barra ligada ao apoio, e a subtração deixa só a reação R_p.

3.7. Cálculo dos Esforços Internos

Depois de resolvido o sistema, os esforços de cada barra saem dos deslocamentos dos seus nós, lidos do vetor global: `u_e = u[element_dofs(e, d)]`. Na treliça e no pórtico, u_e é levado aos eixos locais da barra por $u_{\text{loc}} = T u_e$. Cada tipo de elemento tem a sua função `get_internal_forces`, e o resultado é um conjunto de valores nomeados que o relatório imprime sem precisar conhecer o elemento.

Elemento	**Esforços (eixos locais)**
Treliça	N
Viga	V_i, V_j, M_i, M_j
Pórtico plano	N_i, N_j, V_i, V_j, M_i, M_j

Tabela 2: Esforços calculados por elemento.

Esforço normal. É calculado como na treliça, $N = E\frac{A}{L}(u_j - u_i)$ com u nos eixos locais, positivo em tração. Se houver carga ao longo da barra (q_a), N varia linearmente e esse valor é o da média: $N_i = N + q_a\frac{L}{2}$ e $N_j = N - q_a\frac{L}{2}$.

Flexão. Na viga e no pórtico, integra-se a equação da linha elástica, $EIv'''' = q_t$, com q_t a carga uniforme perpendicular à barra. Adotando $V = EIv'''$ e $M = EIv''$:

$$V(x) = q_t x + c_1, \quad M(x) = \frac{q_t x^2}{2} + c_1 x + c_2$$

$$EIv'(x) = \frac{q_t x^3}{6} + \frac{c_1 x^2}{2} + c_2 x + c_3, \quad EIv(x) = \frac{q_t x^4}{24} + \frac{c_1 x^3}{6} + \frac{c_2 x^2}{2} + c_3 x + c_4$$

As quatro constantes de integração (c_1 a c_4) são obtidas da equação abaixo, implementada na função `integration_constants`. Ela usa os deslocamentos locais das extremidades da barra, v (transversal) e θ (rotação):

$$\begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix} = \begin{pmatrix} \frac{12EI}{L^3} & \frac{6EI}{L^2} & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ -\frac{6EI}{L^2} & -\frac{4EI}{L} & \frac{6EI}{L^2} & -\frac{2EI}{L} \\ 0 & EI & 0 & 0 \\ EI & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} v_i \\ \theta_i \\ v_j - \frac{q_t L^4}{24EI} \\ \theta_j - \frac{q_t L^3}{6EI} \end{pmatrix}$$

4. Validação dos Modelos Estruturais

Os resultados do programa foram comparados com os do Trabalho I e, no pórtico, com o Ftool. Os relatórios abaixo são os arquivos gerados em `outputs/`, sem edição.

4.1. Treliça Bidimensional

Entrada: `inputs/2d_truss_trabalho1.toml` (dados do Exercício I, em kN e m).

```
DIRECT STIFFNESS METHOD - REPORT
================================
(values in the units of the input file)

1. Model
--------
Structure type : 2d_truss
Nodes          : 5
Elements       : 7
DOFs per node  : 2
DOFs total     : 10 (6 free, 4 prescribed)

2. Elements
-----------
elem     i     j            L            E            A  angle [deg]
   1     1     2       2.0000   1.8000e+08       0.0050        0.000
   2     2     3       2.0000   1.8000e+08       0.0050        0.000
   3     1     4       2.0000   1.8000e+08       0.0050       60.000
   4     4     2       2.0000   1.8000e+08       0.0050      -60.000
   5     2     5       2.0000   1.8000e+08       0.0050       60.000
   6     5     3       2.0000   1.8000e+08       0.0050      -60.000
   7     4     5       2.0000   1.8000e+08       0.0050        0.000

3. Applied loads
----------------
F4x             10.0000

4. Prescribed displacements
---------------------------
u1x             0.000000
u1y             0.000000
u3x             0.000000
u3y             0.000000

5. Nodal displacements
----------------------
DOF                  value
u1x           0.000000e+00
u1y           0.000000e+00
u2x           5.555556e-06
u2y          -6.415003e-06
u3x           0.000000e+00
u3y           0.000000e+00
u4x           3.055556e-05
u4y          -4.811252e-06
u5x           1.944444e-05
u5y          -1.603751e-06

6. Support reactions
--------------------
DOF                 value
R1x                -5.0000
R1y                -4.3301
R3x                -5.0000
R3y                 4.3301

7. Internal forces
------------------
elem              N
   1         2.5000
   2        -2.5000
   3         5.0000
   4        -5.0000
   5         5.0000
   6        -5.0000
   7        -5.0000
```

Figura 4: Relatório do programa para a treliça plana.

Os deslocamentos e os esforços normais coincidem com o Trabalho I:

GDL	**Trabalho I** ($\times 10^{-3}$ **mm**)	**Programa** ($\times 10^{-3}$ **mm**)	**Barra**	**N (kN)** **Trabalho I = Programa**
u_{2x}	5.556	5.556	1	$+2.5$
u_{2y}	-6.415	-6.415	2	-2.5
u_{4x}	30.556	30.556	3	$+5.0$
u_{4y}	-4.811	-4.811	4	-5.0
u_{5x}	19.444	19.444	5	$+5.0$
u_{5y}	-1.604	-1.604	6 e 7	-5.0

Tabela 3: Treliça plana: Trabalho I × programa.

4.2. Treliça Espacial

Entrada: `inputs/3d_truss.toml` (Exercício II).

```
DIRECT STIFFNESS METHOD - REPORT
================================
(values in the units of the input file)

1. Model
--------
Structure type : 3d_truss
Nodes          : 4
Elements       : 3
DOFs per node  : 3
DOFs total     : 12 (3 free, 9 prescribed)

2. Elements
-----------
elem     i     j           L           E           A  l, m, n
   1     2     1      3.0000   1.8000e+08      0.0050  0.000, 0.000, 1.000
   2     1     3      3.6056   1.8000e+08      0.0050  0.000, 0.555, -0.832
   3     1     4      4.1231   1.8000e+08      0.0050  0.485, 0.485, -0.728

3. Applied loads
----------------
F1y             10.0000

4. Prescribed displacements
---------------------------
u2x            0.000000
u2y            0.000000
u2z            0.000000
u3x            0.000000
u3y            0.000000
u3z            0.000000
u4x            0.000000
u4y            0.000000
u4z            0.000000

5. Nodal displacements
----------------------
DOF                  value
u1x         -1.302005e-04
u1y          2.052005e-04
u1z          5.000000e-05
u2x          0.000000e+00
u2y          0.000000e+00
u2z          0.000000e+00
u3x          0.000000e+00
u3y          0.000000e+00
u3z          0.000000e+00
u4x          0.000000e+00
u4y          0.000000e+00
u4z          0.000000e+00

6. Support reactions
--------------------
DOF                value
R2x               0.0000
R2y               0.0000
R2z             -15.0000
R3x               0.0000
R3y             -10.0000
R3z              15.0000
R4x               0.0000
```

```
R4y               0.0000
R4z              -0.0000

7. Internal forces
------------------
elem             N
   1       15.0000
   2      -18.0278
   3        0.0000
```

Figura 5: Relatório do programa para a treliça espacial.

Grandeza	Trabalho I	Programa	Unidade
u_{1x}	-0.13	-0.130	mm
u_{1y}	0.205	0.205	mm
u_{1z}	0.05	0.050	mm
N_1	$+15$	$+15.000$	kN
N_2	-18.014	-18.028	kN
N_3	0	0.000	kN

Tabela 4: Treliça espacial: Trabalho I × programa.

A única diferença está na barra 2 (-18.014 contra -18.028 kN) e vem do arredondamento do Trabalho I .

4.3. Viga de Euler-Bernoulli

Entrada: `inputs/2d_beam.toml` (Exercício III, viga biengastada com q, P e M).

```
DIRECT STIFFNESS METHOD - REPORT
================================
(values in the units of the input file)

1. Model
--------
Structure type : 2d_beam
Nodes          : 4
Elements       : 3
DOFs per node  : 2
DOFs total     : 8 (4 free, 4 prescribed)

2. Elements
-----------
elem     i     j           L           E           Iz  angle [deg]
   1     1     2      2.0000   1.8000e+08   2.1000e-06        0.000
   2     2     3      2.0000   1.8000e+08   2.1000e-06        0.000
   3     3     4      3.0000   1.8000e+08   2.1000e-06        0.000

3. Applied loads
----------------
F1y             -0.5000
F1θ             -0.1667
F2y             -0.5000
F2θ              0.1667
F3y            -10.0000
F3θ              2.0000

4. Prescribed displacements
---------------------------
u1y            0.000000
u1θ            0.000000
u4y            0.000000
u4θ            0.000000

5. Nodal displacements
----------------------
DOF                 value
u1y          0.000000e+00
```

```
u1θ          0.000000e+00
u2y         -2.784077e-02
u2θ         -1.940035e-02
u3y         -4.648526e-02
u3θ          8.314437e-03
u4y          0.000000e+00
u4θ          0.000000e+00

6. Support reactions
--------------------
DOF                 value
R1y                5.2857
R1θ                8.6190
R4y                5.7143
R4θ               -9.6190

7. Internal forces
------------------
elem             Vi            Vj            Mi            Mj
   1         5.2857        4.2857       -8.6190        0.9524
   2         4.2857        4.2857        0.9524        9.5238
   3        -5.7143       -5.7143        7.5238       -9.6190
```

Figura 6: Relatório do programa para a viga.

Grandeza	Analítico (Trabalho I)	Rigidez direta (Trabalho I)	Programa
V_a (kN)	5.286	5.286	5.286
M_a (kN · m)	8.619	8.619	8.619
V_d (kN)	5.714	5.714	5.714
M_d (kN · m)	−9.617	−9.619	−9.619
$v(x = 2)$ (mm)	−27.841	−27.84	−27.841
$v(x = 4)$ (mm)	−46.484	−46.485	−46.485

Tabela 5: Viga: Trabalho I × programa.

O programa reproduz o resultado da rigidez direta do Trabalho I. A diferença de 0.002 kN · m para o resultado analítico em M_d vem de arredondamento no cálculo analítico.

4.4. Pórtico Bidimensional

Entrada: `inputs/2d_frame.toml` (Exercício IV).

```
DIRECT STIFFNESS METHOD - REPORT
================================
(values in the units of the input file)

1. Model
--------
Structure type : 2d_frame
Nodes          : 4
Elements       : 3
DOFs per node  : 3
DOFs total     : 12 (6 free, 6 prescribed)

2. Elements
-----------
elem     i     j           L           E           A  angle [deg]
   1     1     2      2.0000   1.8000e+08      0.0050       90.000
   2     2     3      2.0000   1.8000e+08      0.0050        0.000
   3     3     4      2.0000   1.8000e+08      0.0050      -90.000

3. Applied loads
----------------
```

```
F2y             -0.5000
F20             -0.1667
F3y             -0.5000
F30              0.1667

4. Prescribed displacements
---------------------------
u1x             0.000000
u1y             0.000000
u10             0.000000
u4x             0.000000
u4y             0.000000
u40             0.000000

5. Nodal displacements
----------------------
DOF                 value
u1x          0.000000e+00
u1y          0.000000e+00
u10          0.000000e+00
u2x          9.256344e-08
u2y         -1.111111e-06
u20         -1.470187e-04
u3x         -9.256344e-08
u3y         -1.111111e-06
u30          1.470187e-04
u4x          0.000000e+00
u4y          0.000000e+00
u40          0.000000e+00

6. Support reactions
--------------------
DOF                 value
R1x                0.0833
R1y                0.5000
R10               -0.0555
R4x               -0.0833
R4y                0.5000
R40                0.0555

7. Internal forces
------------------
elem             Ni            Nj            Vi            Vj            Mi            Mj
   1        -0.5000       -0.5000       -0.0833       -0.0833        0.0555       -0.1111
   2        -0.0833       -0.0833        0.5000       -0.5000       -0.1111       -0.1111
   3        -0.5000       -0.5000        0.0833        0.0833       -0.1111        0.0555
```

Figura 7: Relatório do programa para o pórtico plano.

Na comparação com o Trabalho I houve divergência. Após uma análise mais criteriosa, concluiu-se que o erro estava no Trabalho I, que adotou $\theta = 90°$ para a barra 3. Como a barra 3 vai do nó 3 (topo) ao nó 4 (base), o ângulo correto é $-90°$. Com a matriz do elemento 3 corrigida, os resultados do Trabalho I passam a coincidir com os do programa. Como verificação independente, a estrutura foi modelada no Ftool:

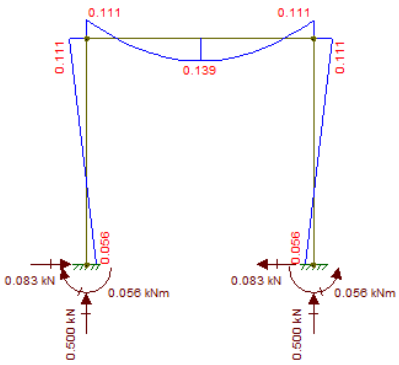


Figura 8: Pórtico no Ftool: momento fletor e reações.

Grandeza	Programa	Ftool
R_{1x} (kN)	0.083	0.083
R_{1y} (kN)	0.500	0.500
$R_{1\theta}$ (kN · m)	−0.056	0.056
M no topo da coluna (kN · m)	0.111	0.111
M no meio do vão (kN · m)	0.139	0.139

Tabela 6: Pórtico: programa × Ftool. Os sinais seguem a convenção de cada programa, e o momento no meio do vão foi obtido de V_i e M_i do relatório.

5. Melhorias Futuras

Para tornar o programa mais robusto, poderiam ser implementados:

- testes automatizados;
- visualização gráfica dos resultados;
- elementos de pórtico tridimensional;
- rótulas internas;
- cargas trapezoidais;
- elementos de casca (*shell*), para analisar estruturas mais gerais.

Há também uma limitação de projeto. O elemento guarda apenas os índices de dois nós, e os GDL são deduzidos do tipo de estrutura. Por isso o programa não analisa estruturas com mais de um tipo de elemento, como pórticos atirantados, nem elementos com mais de dois nós, como barras curvas. Para incluir estas melhorias, o elemento deveria guardar a lista dos seus nós e dos respectivos GDL, e a montagem passaria a percorrer essa lista.

6. Conclusão

Implementou-se, em Julia, um programa de análise pelo Método da Rigidez Direta para treliças, vigas e pórticos planos e treliças espaciais. Ele lê a estrutura de um arquivo TOML, monta a matriz global de forma esparsa, trata apoios, recalques e cargas distribuídas, e devolve deslocamentos, reações e esforços internos em um relatório de texto.

Os resultados coincidem com os do Trabalho I nas treliças e na viga e com o Ftool no pórtico. A comparação também apontou um erro no Trabalho I, na matriz da barra 3 do pórtico. O programa substitui a montagem manual das matrizes, que era a etapa mais sujeita a erros no Trabalho I.